HW2 实验报告

Static Timing Analysis

Tongbin Zhao　学号：2300012831　2026 年 10 月 25 日

1 Problem 1：Elmore delay

对每个节点维护下游总电容 $C_{\downarrow}[v]$ 和延迟 $D[v]$。先后序遍历累加电容，再先序遍历传播延迟。以下伪代码中的 `edge`、`edge` 和 `v` 分别对应题目给出的数据结构 `RctEdge` 和 `RctNode`。

```
def postorder_cap(v):
    C_down[v] = v.cap
    for edge in v.fanout:
        postorder_cap(edge.to)
        C_down[v] += C_down[edge.to]

def preorder_delay(v):
    for edge in v.fanout:
        w = edge.to
        D[w] = D[v] + edge.res * C_down[w]
        preorder_delay(w)

def elmore_delay(root):
    postorder_cap(root)
    D[root] = 0
    preorder_delay(root)
    return C_down, D
```

2 Problem 2: Arrival time and slack

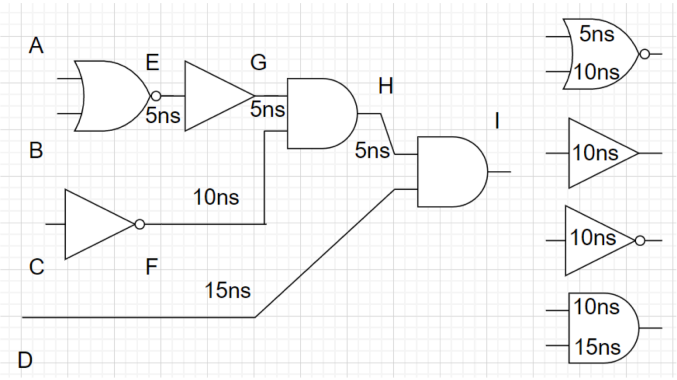


图 1: 题目 Figure 2：组合电路及延迟。图片裁自作业原 PDF。

图右侧标出门输入到输出的延迟，电路内部标注的是对应连线延迟。以每个字母标记处为节点，将连线和下一级门弧的延迟相加，得到下表中的有效边权。

边	$A\to E$	$B\to E$	$C\to F$	$E\to G$	$G\to H$	$F\to H$	$H\to I$	$D\to I$
d / ns	5	10	10	$5+10$	$5+10$	$10+15$	$5+10$	$15+15$

由 $\mathrm{AT}(A)=\mathrm{AT}(B)=\mathrm{AT}(C)=\mathrm{AT}(D)=0$ 前向传播：

$$\mathrm{AT}(E)=\max(0+5,0+10)=10,\qquad \mathrm{AT}(F)=0+10=10,$$
$$\mathrm{AT}(G)=10+15=25,\qquad \mathrm{AT}(H)=\max(25+15,10+25)=40,$$
$$\mathrm{AT}(I)=\max(40+15,0+30)=55.$$

由 $\mathrm{RAT}(I)=50$ 反向传播，得 $\mathrm{RAT}(H)=35$、$\mathrm{RAT}(D)=20$、$\mathrm{RAT}(G)=20$、$\mathrm{RAT}(F)=10$、$\mathrm{RAT}(E)=5$，再传播到主输入。

节点	AT / ns	RAT / ns	slack / ns
A	0	0	0
B	0	-5	-5
C	0	0	0
D	0	20	20
E	10	5	-5
F	10	10	0
G	25	20	-5
H	40	35	-5
I	55	50	-5

关键路径为 $B\to E\to G\to H\to I$，总延迟 $10+15+15+15=55$ ns，相对 50 ns 的约束违反 5 ns。

3 Problem 3: Setup analysis before CPPR

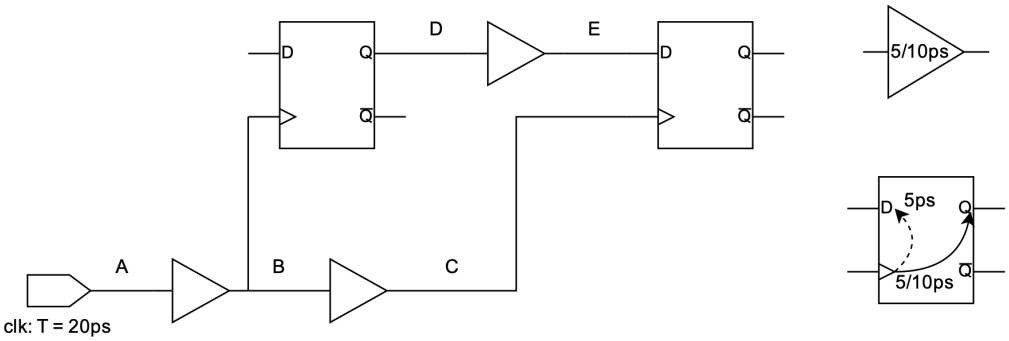


图 2: 题目 Figure 3：同相时钟树及寄存器间数据路径。

A 是时钟根节点，B 是第一级时钟缓冲器输出及发射 FF 的时钟端，C 是第二级时钟缓冲器输出及捕获 FF 的时钟端。图中 D 是发射 FF 的数据输出，E 是捕获 FF 的数据输入。每个缓冲器及 FF 的 clock-to-Q 延迟均为 early/late = 5/10 ps；线延迟为零。

节点	early AT / ps	late AT / ps
A	0	0
B	5	10
C	10	20
D	10	20
E	15	30

capture edge 到达 C 的绝对 early/late 时刻为 $20+10=30$ ps 和 $20+20=40$ ps。

setup 检查使用 late 数据 AT、early capture edge 和 $t_{\text{setup}}=5$ ps：

$$\text{RAT}(E) = T + \text{AT}_{\text{early}}(C) - t_{\text{setup}} = 20 + 10 - 5 = 25 \text{ ps},$$

$$\text{RAT}(D) = \text{RAT}(E) - d^{\text{late}}_{D\to E} = 25 - 10 = 15 \text{ ps}.$$

于是 $\text{slack}(D) = 15 - 20 = -5$ ps，$\text{slack}(E) = 25 - 30 = -5$ ps。两者均违反 setup 约束。

4 Problem 4: Inverted capture clock

将 $B \to C$ 的缓冲器换成反相器后，按照题意，捕获可用时间改为半周期 $T/2 = 10$ ps。时钟路径 early 延迟仍为 $5+5=10$ ps，故

$$\text{RAT}(E) = T/2 + \text{AT}_{\text{early}}(C) - t_{\text{setup}} = 10 + 10 - 5 = 15 \text{ ps}.$$

数据路径未变，$\text{AT}_{\text{late}}(E) = 30$ ps；相应的 CPPR 前 slack 为 -15 ps。

5 Problem 5: Setup timing with CPPR

本题为代码实现，实现后的源码见 `sta.py`。本节给出实现要点、递推关系、复杂度分析及运行结果。

5.1 七个函数及递推关系

函数	实现要点
`topological_order`	Kahn 入度队列，处理所有数据节点；若输出节点数不足则报告环。
`propagate_clock`	从根开始按父子关系累加 early/late 边延迟，不依赖节点 ID 顺序。
`propagate_arrival`	对每个起点单独做最大 AT 传播，记录前驱；不可达值为 `None`。
`propagate_required`	初始化所有 FF 端点约束后逆拓扑取最小值；无约束节点为 $+\infty$。
`compute_cppr`	沿父指针找到两条根路径，比较公共前缀，返回 credit 和最后公共引脚名。
`summarize_endpoints`	枚举可达起点–端点对，修正前后分别选最小 slack，再统计 WNS/TNS。
`trace_critical_path`	在所选起点的前驱表中从端点反向回溯，再反转得到数据路径。

对于起点 s，只初始化该起点：

$$\mathrm{AT}_s(s) = t_{cq}(s) + L(c_s), \qquad \mathrm{AT}_s(v) = \max_{u \to v}\{\mathrm{AT}_s(u) + d_{uv}^{\mathrm{late}}\}.$$

其中 PI/tie 没有 c_s，起点初始化为零。不能先合并所有起点的 AT：不同起点到同一端点共享的时钟路径不同，最大 AT 的起点可能在 CPPR 后不再最坏。相同 AT 的候选前驱选择较小 ID；最终 slack 相同则先按起点 ID、再按端点 ID 排序。

设 $E(c_e)$ 为捕获时钟的 early 到达时间，P_s, P_e 为根到发射/捕获时钟引脚的路径，则

$$\begin{aligned} R_0(e) &= T + E(c_e) - t_{\mathrm{setup}}(e), \\ \Delta(s,e) &= \sum_{a \in \mathrm{prefix}(P_s,P_e)} (d_a^{\mathrm{late}} - d_a^{\mathrm{early}}), \\ R_1(s,e) &= R_0(e) + \Delta(s,e), \qquad S_1(s,e) = R_1(s,e) - \mathrm{AT}_s(e). \end{aligned}$$

`propagate_required` 返回的是基准 R_0 的反向传播；CPPR 后 RAT 与起点有关，因此修正保存在每个 pair 中，不合并成单个节点 RAT。

5.2 最坏起点改变的实际端点

端点为 `dpath.b_reg.out[5]$_DFFE_PP_/D`，记两个候选起点为：

- L1：`dpath.a_reg.out[7]$_DFFE_PP_/QN`。
- L2：`dpath.a_reg.out[9]$_DFFE_PP_/QN`。

起点	AT / ns	slack 前 / ns	CPPR / ns	slack 后 / ns
L1	0.312791381	-0.005704594	0.009021508	0.003316913
L2	0.309360046	-0.002273259	0.003627541	0.001354282

L1 的公共时钟引脚为 `clkbuf_2_0__f_clk/Y`；L2 的公共时钟引脚为 `clkbuf_0_clk/Y`。修正前，L1 的 slack 更小；修正后，L1 共享的时钟前缀较长，得到更大的 credit，最坏起点转为 L2。该端点的最坏 slack 从 -0.005704594 ns 变为 $+0.001354282$ ns。若只修正原来的 L1 并继续把它作为最坏起点，会误报为 $+0.003316913$ ns，高估该端点的裕量。这验证了逐起点保留 AT 并重新选取最坏路径的必要性。

5.3 独立运行 OpenSTA 比较 CPPR

使用 `compare_opensta.py` 另起 OpenSTA 进程，读入相同网表和库。读完原 SDC 后执行 `set_cmd_units -time ns` 并将时钟周期重设为 0.28 ns，避免将 ASAP7 的 ps 库单位直接误作 ns。原 SDC 的 I/O 约束予以保留。采用传播时钟及 on-chip variation 分析，核心对照设置为：

```
set_timing_derate -early -cell_delay 0.9
set_timing_derate -late -cell_delay 1.1
sta::set_crpr_mode same_transition
```

脚本将 `sta::set_crpr_enabled` 分别设为 0 和 1，固定发射 FF 时钟引脚、数据输出引脚和接收 D 引脚，输出 `full_clock_expanded` 文本与 JSON；比较实际报告中的 credit，同时检查两条原生时钟路径的最后公共引脚与本实现一致。所比较的三对路径为：

- P1：`dpath.a_reg.out[13]$_DFFE_PP_/QN` → `dpath.a_reg.out[7]$_DFFE_PP_/D`。
- P2：`dpath.a_reg.out[7]$_DFFE_PP_/QN` → `dpath.b_reg.out[5]$_DFFE_PP_/D`。
- P3：`dpath.a_reg.out[9]$_DFFE_PP_/QN` → `dpath.b_reg.out[5]$_DFFE_PP_/D`。

路径	本实现 CPPR / ns	OpenSTA CPPR / ns	绝对误差 / ns
P1	0.003627541	0.003627542	5.04e-10
P2	0.009021508	0.009021506	1.67e-09
P3	0.003627541	0.003627542	5.04e-10

三对路径均通过 2×10^{-8} ns 的绝对误差门限，最大实际误差约为 1.67×10^{-9} ns。浮点计算与文本舍入可解释此量级的差异。OpenSTA 原生 slack 与本实现不同：本题将各级数据 rise/fall 弧独立取最大，而 OpenSTA 保留跃迁一致性，且使用原生 SDC 约束。故此处验证的是 CPPR credit，不能用原生绝对 slack 替代本题的计算结果。

附录：AI 使用记录

本次实验中使用 OpenAI Codex 完成了一些辅助工作。实验过程中使用的 prompt 如下：

1. 帮我整理一下 Problem 1-4 中的文字和公式排版，部分地方可能 latex 语法不正确，帮我改正。
2. 帮我安装 OpenSTA 到我的 Mac 上，并写一个 Python 脚本调用 OpenSTA 进行静态时序分析，与 `sta.py` 的运行结果进行对照。
3. 检查一下 `sta.py` 中的代码是否有错误，给出修改建议。
4. 帮我写一下 Problem 5 的报告，实验结果从 `run_gcd/result.json` 中读取，给出实现要点、运行结果和对照 OpenSTA 的验证。
5. 帮我整理一下需要提交的代码和报告，放到 `handin` 目录中，目录结构按照 `problem.pdf` 的提交要求组织，并写一个 `README.md` 介绍目录结构以及如何复现实验结果